



INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Asignatura:

Programación Avanzada

Proyecto 1

Sistema de Gestión de Productos e Inventario

Entrega 3 - Borrador

Profesor:

Ing. Juan Jose Vanzetti

Integrantes:

Albarracin, Trinidad	trinialbarracin13@gmail.com	15660
Magallanes, Agustin	agus57cr@gmail.com	16045
Mansilla, Santiago	santiagomansilla025@gmail.com	15921
Nardi, Elisa Maria	unielisanardi@gmail.com	16136
Olguin, Juan Cruz	olguinjuancruz@gmail.com	17241

Fecha de Entrega: 02/09/2026

Índice

Auditoría de Consistencia: Dominio vs. Implementación	2
1. La entidad Producto no tiene métodos	2
2. estaBajoMinimo() no existe en ningún lugar (ni back ni front)	7
3. Los eventos de dominio StockBajo y StockActualizado no existen	10
4. MovimientoStock existe como tabla pero está vacío funcionalmente	11
5. Proveedor no existe en el análisis	14
6. El sistema de roles de usuario no está documentado en el análisis de dominio	16
7. No se pueden loguear nuevos usuarios	18
Backlog de Evolución de la Documentación de Dominio	19
Cambios propuestos	20

Auditoría de Consistencia: Dominio vs. Implementación

1. La entidad Producto no tiene métodos

Categoría D: Regla mal ubicada (antipatrón anémico)

En el archivo [producto.entity.ts](#) se puede ver que esta entidad no cuenta con ningún tipo de método; es un contenedor de datos, por lo cual no cumple con lo que una entidad debe ser ya que estas deben tener sus comportamientos y encapsular sus reglas de negocio. Aquí se puede ver en la estructura de la entidad que está solo guarda datos.

`@Entity('producto')`

```
export class Producto {
  @ApiProperty()
  @PrimaryGeneratedColumn()
  id: number;

  @ApiProperty()
  @Column({ type: 'text' })
  denominacion: string;

  @Index()
  @Column({ type: 'varchar', length: 255, nullable: true })
  codigoProveedor?: string | null;

  @Column({ type: 'text', nullable: true })
  codigoBarra?: string | null;

  // ===== PROVEEDOR =====
  @ManyToOne(() => Proveedor, (pro) => pro.proveedoresOperacion, {
    eager: true,
  })
  @JoinColumn({ name: 'proveedor_id' })
  @Index()
  proveedor: Proveedor;

  @Column({ type: 'int', nullable: true })
  proveedorId?: number;

  /*
  Nota: No usar el enum alciculta iva en @Column
      sino no anda el importar precios
  */
  @PorcentajeColumn(21.0)
  alicuotalva: Alicuotalva;

  // Stock: cantidades reales, admite fracciones (1.5 kg, 0.25 lts)
  @CantidadColumn()
```

```
stock: number;

@Column('boolean', { default: false })
utilizaStockMinimo: boolean;

@Column('boolean', { default: false })
utilizaStockMinimoPorEmpresa: boolean;

@CantidadColumn()
stockMinimo: number;

@MonetarioColumn()
costo?: number;

@MonetarioColumn()
costoDolar?: number;

/*
Ultima cotizacion dolar por el cambio de precio si producto posee costo dolar
*/
@MonetarioColumn()
cotizacionDolar?: number;
//se utiliza en las importaciones;

@MonetarioColumn()
precioDolar?: number;
// Precio de venta

@MonetarioColumn()
precio?: number;

@PorcentajeColumn()
porcentaje?: number;

@Column({ type: 'timestamp', nullable: true })
fechaCosto?: Date;

@Column('boolean', { default: false })
costoEnDolar?: boolean;

@Column({ type: 'timestamp', nullable: true })
fechaCostoDolar?: Date;

@Column('boolean', { default: false })
destacado?: boolean;

@Column('boolean', { default: false })
envioGratis?: boolean;

@Column({ type: 'text', nullable: true })
```

```
observacion?: string;

@CreateDateColumn()
createdAt: Date;

@UpdateDateColumn()
updatedAt: Date;

@Column({ type: 'timestamp', nullable: true })
@index()
deletedAt?: Date;

@ManyToOne(() => Usuario)
@JoinColumn({ name: 'usuario_created_id' })
usuarioCreated: Usuario;

@ManyToOne(() => Usuario)
@JoinColumn({ name: 'usuario_updated_id' })
usuarioUpdated: Usuario;

@ManyToOne(() => Usuario)
@JoinColumn({ name: 'usuario_deleted_id' })
usuarioDeleted: Usuario;

// ===== LINEA =====
@ManyToOne(() => Linea, (linea) => linea.productos)
@JoinColumn({ name: 'linea_id' })
linea: Linea;

@Column({ type: 'int', nullable: true })
lineaId?: number;

// ===== MARCA =====
@ManyToOne(() => Marca, (marca) => marca.productos)
@JoinColumn({ name: 'marca_id' })
marca: Marca;

@Column({ type: 'int', nullable: true })
marcaId?: number;

@Column({ default: false })
utilizaPack: boolean;

@Column({ type: 'int', nullable: true })
cantidadPorPack: number | null;

@Column({ type: 'text', nullable: true })
imagen?: string;
```

```
@Column({ type: 'text', nullable: true })
ubicacion?: string;

@ManyToOne(() => Producto, (producto) => producto.productosOperacion)
productosOperacion: ProductoOperacion;

@Column({ type: 'int', default: 0 })
sistema: number;

@Column({ type: 'text', nullable: true })
codigoReferencia?: string | null;
}
```

Esta es una de las principales razones por las cuales la entidad actúa como un modelo anémico ya que no presenta ningún tipo de comportamiento; es como un DTO disfrazado.

Los métodos están en distintos archivos que deben cumplir con otras funciones no con realizar métodos del dominio, como por ejemplo, dentro de [producto.service.ts](#), encontramos el siguiente método para crear productos, y otro llamado `ajustarStockInterno()` el cual actualiza el stock, pero tiene la validación de este comentada.

En este archivo solo deberíamos encontrar servicios de aplicación.

Aquí uno de los métodos encontrados dentro del archivo:

```
async create(dto: CreateProductoDto) {

  this.logger.log(
    `Creando un nuevo ${this.ENTITY_NAME} con denominación: ${dto.denominacion} a: ${dto.denominacion}`,
  );

  // Orquestar todas las validaciones

  const { marca, linea, usuario } =
    await this.validarYPrepararCreacion(dto);

  const entity = await this.repository.create(
    dto,
    linea,
    marca,
    usuario,
  );
}
```

```
return MessageFrontUtils.createSimple(  
  `${this.ENTITY_NAME}`,  
  entity.denominacion,  
  'creada',  
);
```

En [producto.persistence-adapters.ts](#) encontramos una vez más un error muy grave, esta hardcoded la validación de stock dentro de este archivo que es para SQL directo.

```
if (conStock) {  
  query.andWhere('producto.stock > 0');  
}
```

En [product.mapper.ts](#) se encuentra erróneamente gran parte de la lógica de dominio. En este caso, aquí deberíamos encontrar lo correspondiente a la capa de infraestructura.

En esta encontramos métodos para calcular precios con IVA

```
static toBusquedaDto(entity: Producto): GetProductoDto {  
  const precio = entity.precio ?? 0;  
  const alicuota = entity.alicuotalva ?? 0;  
  return {  
    id: entity.id,  
    denominacion: entity.denominacion,  
    observacion: entity.observacion ?? '',  
    codigoProveedorDenominacion:  
      entity.codigoProveedor + ' - ' + entity.denominacion,  
    codigoProveedor: entity.codigoProveedor ?? '',  
    proveedor: '',  
    stock: entity.stock,  
    alicuota: alicuota,  
    costo: entity.costo ?? 0,  
    precio: precio,  
    precioConIva: +(precio * (1 + alicuota / 100)).toFixed(2),  
    ubicacion: entity.ubicacion ?? '',  
    utilizaStockMinimo: entity.utilizaStockMinimo,
```

También en el mismo archivo nos encontramos con un método estático el cual actualiza costos.

```
static mapPrecios(  

```

```
entity: Producto,  
dto: UpdatePrecioDto,  
usuario: Usuario,  
): void {  
  entity.costo = dto.costo;  
  entity.costoDolar = dto.costoDolar;  
  entity.cotizacionDolar = dto.cotizacionDolar;  
  
  entity.fechaCostoDolar = new Date();  
  entity.fechaCosto = new Date();  
  entity.usuarioUpdated = usuario;  
}
```

Por último nos encontramos que no hay ningún método que calcule o aplique el porcentaje que se encuentra almacenado como propiedad del producto en [producto.entity.ts](#) además, podemos ver que `esProductoDestacado()` aparece como solo como una propiedad pública sin ningún tipo de encapsulación.

Impacto en este proyecto: **ALTO**

Tenemos la lógica de dominio distribuida en tres capas diferentes del proyecto (service, mapper y repository). La entidad no realiza ningún comportamiento, solo es un contenedor de datos, tampoco tiene encapsulación.

Esto lleva a los productos a violar el DDD por completo.

Este problema se repite en otras entidades como *usuario*, *cliente*, *marca* y *línea*.

2. `estaBajoMinimo()` no existe en ningún lugar (ni back ni front)

Categoría: A — Documentado pero no implementado

La búsqueda en todo el proyecto de `estaBajoMinimo`, `stockBajo`, `stock_bajo`, `stock-bajo` no devuelve ningún resultado. El comportamiento de alerta de stock está completamente ausente de la implementación. El front no muestra ningún indicador visual diferenciado cuando `stock <= stockMinimo`, aunque los campos `stockMinimo` y `utilizaStockMinimo` sí existen en el modelo.

El problema tiene tres capas que tenemos que atacar en orden:

Capa 1 — Back: la entidad Producto no tiene el método

Archivo: `producto.entity.ts`

La entidad tiene stock, stockMinimo y utilizaStockMinimo como campos, pero no tiene ningún método. Es un objeto TypeORM puro, sin comportamiento. Aquí debería vivir estaBajoMinimo().

Capa 2 — Back: el mapper no expone el resultado al DTO

Archivo: producto.mapper.ts → método toBusquedaDto()

El mapper ya envía utilizaStockMinimo y stockMinimo al front, pero no calcula ni agrega un campo stockBajo: boolean que el front pueda consumir directamente.

Capa 3 — Front: la tabla y la card no muestran ningún indicador visual

Archivos:

- interfaces-producto.tsx → ConsultarProducto no tiene el campo stockBajo
- datos-tabla.tsx → la columna "Stock" no tiene ningún color diferenciado por mínimo
- datos-card.tsx → el stock solo se pone rojo si es < 0, no si es <= stockMinimo

Capa 1 — Back: agregar el método en la entidad

Categoría D — Regla mal ubicada (antipatrón anémico) La regla existe conceptualmente en el análisis, pero no vive en la entidad de dominio donde debería estar.

Se agrega estaBajoMinimo() directamente en producto.entity.ts, después de los campos stock, utilizaStockMinimo y stockMinimo. La regla vive en la entidad porque es ahí donde viven los datos que la alimentan, tal como establece el análisis en la sección 5.2.

```
typescript
@Column({ type: 'text', nullable: true })
codigoReferencia?: string | null;
}
```

No tiene ningún método

Capa 2 — Back: exponer el resultado en el DTO y en el mapper

mapper no expone stockBajo **Categoría A — Documentado pero no implementado**

El análisis documenta que estaBajoMinimo() debe detectar el estado, pero ni el método ni su resultado aparecen en ningún lugar de la cadena back→front.

El archivo es producto.mapper.ts. El problema está en el método toBusquedaDto() — ya envía utilizaStockMinimo y stockMinimo al front, pero nunca calcula ni agrega stockBajo.

```
// ANTES — los datos están pero la comparación no se expone:  
utilizaStockMinimo: entity.utilizaStockMinimo,  
  
stockMinimo: entity.stockMinimo,
```

Acá se evidencia que el mapper enviaba los datos al front pero nunca calculaba ni exponía si el producto estaba en alerta.

Capa 3 — Front: la tabla y la card no muestran ningún indicador visual

front no muestra el indicador **Categoría C — Existe en el back pero no se expone en el front** Los campos stockMinimo y utilizaStockMinimo llegan al front, pero la UI nunca los usa para mostrar ninguna alerta visual.

Archivo 1: interfaces-producto.tsx

La interfaz ConsultarProducto no tiene el campo stockBajo. El front no tiene forma de recibir ni usar ese dato aunque el back lo enviara.

```
export interface ConsultarProducto {  
  id: number;  
  denominacion: string;  
  codigoProveedor: string;  
  codigoReferencia: string;  
  stock: number;  
  precio: number;  
  // ... no existe stockBajo en ningún lado
```

Archivo 2: datos-card.tsx

El stock solo se colorea de rojo si es negativo. No hay ninguna condición para stockMinimo:

```
<p className={`text-sm font-medium ${  
  producto.stock < 0 ? "text-red-600" : "text-gray-800"  
}`}>  
  {producto.stock}  
</p>
```

Archivo 3: consultar-producto.tsx

La columna Stock no tiene `formatFunction` que compare contra `stockMinimo`. Solo muestra el número:

```
{
  header: "Stock",
  accessor: "stock",
  // sin formatFunction, sin badge, sin color por mínimo
}
```

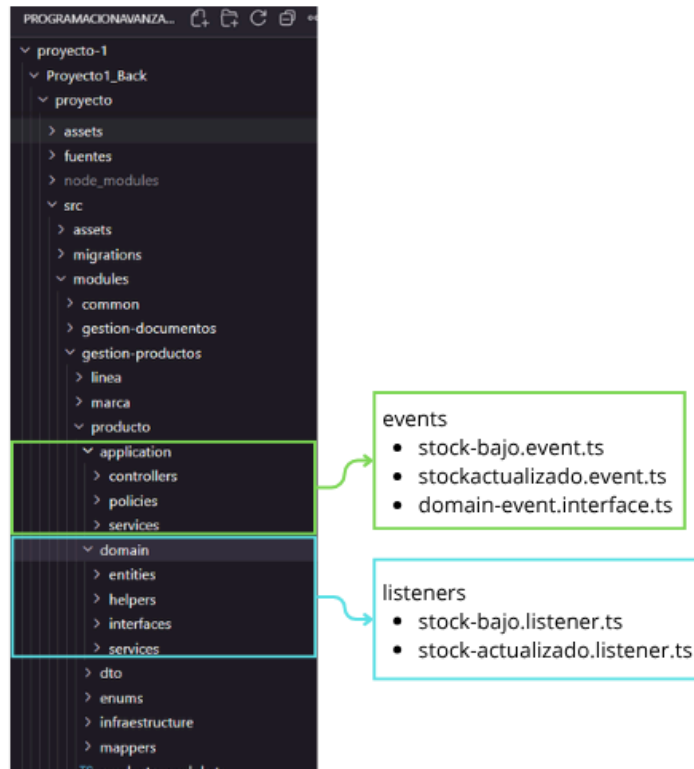
La columna no tiene `formatFunction` definida — solo declara `header` y `accessor`, sin ninguna lógica visual para stock crítico.

Impacto: ALTO -> La regla de negocio 5.2 establece que si $\text{stockActual} \leq \text{stockMinimo}$ el producto entra en estado de alerta, y que esta regla debe vivir en `Producto` a través del método `estaBajoMinimo()`, separando la detección del problema de la reacción ante él. Sin esta implementación, el usuario no tiene ninguna forma de detectar stock crítico salvo comparar los dos números manualmente en la tabla. Es además la base sobre la que deberían dispararse los eventos de dominio `StockBajo` documentados en la sección 7 del análisis — sin `estaBajoMinimo()`, esos eventos tampoco pueden existir.

3. Los eventos de dominio `StockBajo` y `StockActualizado` no existen

Categoría A: Documentado pero no implementado

Estos eventos se encuentran documentados pero no se implementaron en el código. En la siguiente imagen se muestra en qué partes del proyecto corresponde que estos estén implementados.



Impacto en este proyecto: **ALTO**

Debemos hacer estas modificaciones e implementar estos eventos ya que de lo contrario seguiremos intentando realizar las búsquedas de StockBajo, StockActualizado, DomainEvent no tendremos respuesta. Por ello debemos generar los componentes necesarios para generar este mecanismo ya documentado.

4. MovimientoStock existe como tabla pero está vacío funcionalmente

Categoría: E — Deuda técnica reconocida

Dónde está en el código

Todo el módulo vive en una única carpeta:

Proyecto1_Back/proyecto/src/modules/gestion-productos/producto-operacion/

- entities/producto-operacion.entity.ts ← la tabla en base de datos
- dto/create-producto-operacion.dto.ts ← DTO de creación
- dto/update-producto-operacion.dto.ts ← DTO de actualización
- producto-operacion.service.ts ← lógica de negocio
- producto-operacion.controller.ts ← endpoints REST
- producto-operacion.module.ts ← módulo NestJS

Qué tiene la entidad en base de datos

La tabla producto_operacion existe con estos campos:

```
@Entity('producto_operacion')
export class ProductoOperacion {
  @PrimaryGeneratedColumn()
  id: number;
  @ManyToOne(() => Producto, (pro) => pro.productosOperacion, { eager: true })
  producto: Producto;
  @Column()
  operacionId: number;
  @Column()
  tipoOperacion: string; // Ej: 'compra-producto', 'venta-servicio', etc.
  @Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP' })
  creadoEn: Date;
```

Qué le falta comparado con el análisis

El análisis define que MovimientoStock debe tener: id, productold, tipoMovimiento, cantidad, fecha y motivo. Comparando con lo que existe:

Atributo del análisis	¿Existe en el código?
id	Sí
productold	Sí (como relación producto)
tipoMovimiento	Existe como tipoOperacion (string libre, sin enum ni validación)
fecha	Existe como creadoEn (solo timestamp automático de creación)
cantidad	No existe
motivo	No existe

Los dos campos más críticos, cantidad y motivo, están completamente ausentes de la entidad y de todos los DTOs.

El servicio no hace nada real

El archivo producto-operacion.service.ts tiene cinco métodos que devuelven strings de texto fijo, sin ninguna lógica implementada:

```
@Injectable()
export class ProductoOperacionService {
  create(createProductoOperacionDto: CreateProductoOperacionDto) {
    return `This action adds a new productoOperacion`;
  }
  findAll() {
    return `This action returns all productoOperacion`;
  }
  findOne(id: number) {
    return `This action returns a #${id} productoOperacion`;
  }
  update(id: number, updateProductoOperacionDto: UpdateProductoOperacionDto) {
    return `This action updates a #${id} productoOperacion`;
  }
  remove(id: number) {
    return `This action removes a #${id} productoOperacion`;
  }
}
```

Son los métodos que NestJS genera automáticamente al crear un módulo con su CLI, nunca fueron reemplazados por lógica real.

El DTO está vacío

```
export class CreateProductoOperacionDto {}
```

No valida nada, no recibe nada. El DTO de actualización hereda de este, por lo que también está vacío. El controller tiene endpoints REST expuestos (POST /producto-operacion, GET /producto-operacion, etc.) pero todos devuelven los strings placeholder del servicio.

El módulo no está conectado con nada

producto-operacion.module.ts no importa TypeORM, no tiene repositorio, no está conectado a ninguna base de datos. Es un módulo aislado que no puede leer ni escribir en la tabla producto_operacion aunque la tabla exista físicamente.

La consecuencia concreta

El análisis establece en la regla 5.3 que todo ajuste de stock debe registrar un motivo obligatorio y quedar guardado como un MovimientoStock. En el código, cuando se ajusta el stock de un producto (método ajustarStockInterno del ProductoService), el sistema actualiza el número directamente en la tabla producto y no llama en ningún momento al módulo producto-operacion. Es decir: los movimientos de stock ocurren, el stock cambia, pero no queda ningún registro de qué pasó, cuánto cambió ni por qué motivo.

Qué se debería agregar al análisis de dominio

Este hallazgo no requiere cambiar el análisis de dominio. El análisis ya describe correctamente cómo debería funcionar MovimientoStock. Lo que corresponde es reconocerlo como deuda técnica documentada: el módulo fue creado como estructura base pero su implementación quedó pendiente. El análisis podría agregar una nota en la sección 4.2:

- Estado de implementación: la entidad MovimientoStock existe en base de datos con estructura parcial. Los atributos cantidad y motivo definidos en este análisis aún no están implementados. El servicio asociado no tiene lógica real. Implementar esta entidad completa es prerequisite para cumplir la regla de trazabilidad de stock (sección 5.3) y la regla de motivo obligatorio en ajustes (sección 5.4).

5. Proveedor no existe en el análisis

Categoría B — Implementado pero no documentado

Qué encontró el código:

El sistema tiene implementado un módulo completo de Proveedor que el análisis de dominio nunca menciona, no define en el glosario, y no incluye en ningún diagrama. Sin embargo, Proveedor tiene presencia real y concreta en el modelo de Inventario que está directamente relacionado con Producto.

Dónde está en el código

El módulo de Proveedor vive en:

Proyecto1_Back/proyecto/src/modules/organizacion/proveedor/

— domain/entities/proveedor.entity.ts	← la entidad
— application/controllers/	← endpoints REST
— application/services/	← lógica de aplicación
— dto/	← DTOs de entrada y salida
— infraestructure/	← repositorio
— mappers/	← transformaciones

La conexión con Producto está

en:

Proyecto1_Back/proyecto/src/modules/gestion-productos/producto/domain/entities/producto.entity.ts

Y la regla de unicidad del código interno en:

Proyecto1_Back/proyecto/src/modules/gestion-productos/producto/infraestructure/validadores/producto-uniqueness.validator.ts

Qué tiene la entidad Proveedor

La entidad proveedor.entity.ts tiene los siguientes atributos:

- id
- codigoProovedor: Código interno del proveedor como empresa
- denominacion: Nombre comercial
- denominacionAfip: Nombre fiscal registrado en AFIP
- cuit
- domicilio
- condicionIva
- esProveedorMateriaPrima: flag para clasificar el tipo de proveedor
- esProveedorGastos Flag para clasificar el tipo de proveedor
- proveedoresOperacion: Historial de operaciones realizadas con este proveedor
- Auditoría: createdAt, updatedAt, deletedAt, usuarios de cada acción

Es una entidad con ciclo de vida propio: se puede crear, editar, eliminar (soft delete) y auditar. Tiene sus propios endpoints REST bajo el controller @Controller('proveedor').

Cómo se conecta con Producto

En producto.entity.ts, el Producto tiene:

```
// ===== PROVEEDOR =====
@ManyToOne(() => Proveedor, (pro) => pro.proveedoresOperacion, {
  eager: true,
})
@JoinColumn({ name: 'proveedor_id' })
@index()
proveedor: Proveedor;

@Column({ type: 'int', nullable: true })
proveedorId?: number;
```

```
@Index()
@Column({ type: 'varchar', length: 255, nullable: true })
codigoProveedor?: string | null;
```

```
@Column({ type: 'text', nullable: true })
codigoReferencia?: string | null;
```

-> codigoProveedor en Producto no es lo mismo que codigoProveedor en Proveedor: el primero es el código con el que ese proveedor identifica a ese producto en su catálogo; el segundo es el código interno del proveedor como organización.

Además, `codigoProveedor` del `Producto` tiene una regla de unicidad implementada en `ProductoUniquenessValidator.validarCodigoProveedorUnico()`: si ya existe otro producto activo con ese código, el sistema lanza un `ConflictException` y rechaza la operación. El `codigoReferencia` en cambio no tiene restricción de unicidad y funciona como alias de búsqueda secundario.

La búsqueda rápida de productos usa ambos códigos en paralelo:

en

`proyecto\src\modules\gestion-productos\producto\infraestructure\repositories\producto.persistence-adapters.ts`

```
if (codigo) {
  if (exacto) {
    // Exacto solo en los códigos
    query.andWhere(
      "(producto.codigoProveedor = :codigo OR producto.codigoReferencia = :codigo)",
      { codigo },
    );
  } else {
    // Parcial en códigos y denominación
    query.andWhere(
      `
      producto.codigoProveedor LIKE :codigo OR
      producto.codigoReferencia LIKE :codigo OR
      producto.denominacion LIKE :codigo
      `,
      { codigo: `%${codigo}%` },
    );
  }
}
```

Por qué es un problema para el análisis DDD

El análisis menciona "Devolución a proveedor" como uno de los tipos de `MovimientoStock`, pero nunca define qué es un `Proveedor`. Cualquier persona que lea solo el análisis no sabe si `Proveedor` es un campo de texto libre, una entidad, o algo que pertenece a otro contexto. En el código es una entidad completa con su propio módulo, su propio ciclo de vida y una relación directa con el `Producto`.

6. El sistema de roles de usuario no está documentado en el análisis de dominio

Categoría B — Implementado pero no documentado

Qué encontró el código

El análisis de dominio no menciona en ningún momento que el sistema tiene usuarios y roles. Sin embargo, el código implementa un sistema completo de autenticación y autorización con 8 roles distintos que condicionan qué puede hacer cada persona en el sistema. Cada endpoint del back y cada sección del front tiene definido qué roles pueden acceder.

Cuántos roles hay y cuáles son

En el front, el enum Rol está definido en:

Proyecto1_Front/src/interfaces/generales/interfaces-generales.tsx

```
export const Rol = {  
  ADMINISTRADOR: 1,  
  EMPLEADO: 2,  
  REPOSITOR: 3,  
  VENDEDOR: 4,  
  ROOT: 5,  
  REPARTIDOR: 6,  
  COBRADOR: 7,  
  COBRADOR2: 8,  
};
```

En el back, el seed de roles que los crea en la base de datos está en:

Proyecto1_Back/proyecto/src/modules/common/seed/seed-usuario/[seed-usuario.service.ts](#) Y los roles que efectivamente se crean son:

```
async seedRol() {  
  /* ojo no cambiar el orden */  
  const entryData = [  
  
    { denominacion: 'Administrador' },  
    { denominacion: 'Empleado' },  
    { denominacion: 'Repositor' },  
    { denominacion: 'Vendedor' },  
    { denominacion: 'Admin' },  
    { denominacion: 'Repartidor' },  
    { denominacion: 'Cobrador' },  
  
  ];  
}
```

La entidad Rol vive en:

Proyecto1_Back/proyecto/src/modules/gestion-usuario/rol/domain/entities/rol.entity.ts

```
@Entity('rol')
export class Rol {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ type: 'varchar', length: 255, unique: true })
  denominacion: string;

  @Column({ type: 'text', nullable: true })
  observacion?: string;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;

  @DeleteDateColumn({ nullable: true })
  deletedAt?: Date;

  @Column({ type: 'int', nullable: true })
  usuarioCreatedId?: number;

  @Column({ type: 'int', nullable: true })
  usuarioDeletedId?: number;

  @Column({ type: 'int', nullable: true })
  usuarioUpdatedId?: number;
}
```

El problema concreto: los roles no tienen una definición centralizada

1. "Root" vs "Admin": En el back, varios endpoints dicen que solo puede acceder alguien con el rol llamado Root. Sin embargo, cuando el sistema crea los roles en la base de datos, ese rol no se llama Root sino Admin. Son dos nombres distintos para lo que debería ser el mismo rol. En la práctica eso significa que si un usuario tiene el rol Admin asignado, el sistema le va a negar el acceso a esas pantallas porque el nombre no coincide exactamente con el que el endpoint espera.
2. "COBRADOR2": En el front existe un rol llamado COBRADOR2 con el número 8 definido en el código. Pero ese rol nunca fue creado en la base de datos del back y ninguna pantalla ni ningún endpoint lo usa para controlar accesos. Existe escrito en el código pero no tiene ningún efecto real en el sistema, nadie puede tenerlo asignado y nadie lo verifica.

Ambos casos son consecuencia directa de no tener los roles definidos en el análisis de dominio. Sin una lista oficial que diga "los roles del sistema son exactamente estos, con estos nombres", cada parte del sistema los fue escribiendo a su manera. Si esa lista hubiera existido desde el principio, back y front habrían usado los mismos nombres desde el día uno.

7. No se pueden loguear nuevos usuarios

El registro de nuevos usuarios está dentro del código, tiene su correspondiente endpoint en el archivo [auth.controller.ts](#)

```
@Post('registrar')
async register(@Body() registrarUsuarioDto: RegistrarUsuarioDto): Promise<Usuario> {
```

```
return this.authService.registrarUsuario(registrarUsuarioDto);  
}
```

Pero el registro no está disponible ya que una vez registrado y hasheada la contraseña el login no verifica si el usuario es activo. Se debe agregar esa validación en [auth.service.ts](#). El registro con google si funciona aunque con el mismo inconveniente de validación. Además lo que ocurre al crear este nuevo usuario es el método que debería crear el usuario en realidad crea una promesa pendiente no un nuevo usuario. Por último también hay que destacar que no existe un mecanismo de renovación de tokens y que el JWT refresh se realiza cada 7 días.

En síntesis si se podrían loguear usuarios pero faltan mecanismos o se deben corregir para que se puedan registrar correctamente nuevos usuarios, además de corregir elementos de seguridad para las sesiones de estos usuarios en cuanto a la revocación de tokens y el refresh de JWT.

Backlog de Evolución de la Documentación de Dominio

#	Cambio propuesto	Tipo	Basado en qué hallazgo	Impacto	Esfuerzo	Prioridad resultante
1	Aclarar en el Análisis de Dominio que la entidad Producto debe encapsular sus comportamientos y reglas de negocio.	Aclaración	1 – La entidad Producto no tiene métodos y la lógica de dominio se encuentra distribuida en otras capas.	Alto	Bajo	Hacer ya
2	Incorporar Proveedor como entidad del dominio y documentar su relación con Producto.	Nuevo agregado/entidad	5 – Proveedor existe en la implementación pero no está contemplado en el Análisis de Dominio.	Alto	Bajo	Hacer ya

3	Documentar la diferencia entre Código Interno del producto y Código de Referencia, incluyendo la regla de unicidad del Código Interno.	Aclaración / Nueva regla de negocio	5 – La implementación diferencia ambos códigos y valida la unicidad del Código Interno, pero el dominio no lo contempla.	Alto	Bajo	Hacer ya
4	Incorporar Usuario y Rol al dominio y definir de forma centralizada cuáles son los roles reconocidos por el sistema.	Nuevo agregado/entidad	6 – El sistema de usuarios y roles está implementado pero no documentado y existen inconsistencias entre Root, Admin y COBRADOR2.	Alto	Alto	Planificar
5	Documentar las reglas de autorización que determinan qué operaciones puede realizar cada rol.	Nueva regla de negocio	6 – El código restringe operaciones según el rol, pero estas reglas no aparecen en el Análisis de Dominio.	Alto	Alto	Planificar
6	Incorporar una aclaración sobre el estado actual de implementación de MovimientoStock, indicando que su implementación está incompleta respecto de lo definido en el dominio.	Aclaración	4 – MovimientoStock existe parcialmente, pero no posee cantidad ni motivo y su servicio no implementa comportamiento real.	Bajo	Bajo	Si sobra tiempo

7	Aclarar las reglas de registro y habilitación de usuarios, incluyendo las condiciones que debe cumplir un usuario para poder autenticarse.	Aclaración / Nueva regla de negocio	7 – Existen mecanismos de registro y autenticación, pero se detectaron inconsistencias respecto del estado activo del usuario y la creación de nuevos usuarios.	Alto	Alto	Planificar
----------	--	-------------------------------------	---	------	------	------------

Cambios propuestos

1. Incorporación de los roles al lenguaje ubicuo

Roles de usuario

Los roles de usuario representan los distintos niveles de acceso y responsabilidad que pueden tener los usuarios dentro del sistema. Cada rol determina las operaciones que el usuario puede realizar sobre los distintos elementos del sistema.

Los roles contemplados son:

Rol	Qué puede hacer
Root	Usuario con acceso completo al sistema. Es el único rol autorizado para eliminar Provincias. Puede crear, editar y eliminar Productos, Clientes, Proveedores, Personal, Marcas y Líneas. Posee acceso a todas las secciones del sistema, incluyendo Configuración, Proveedor y Personal.
Administrador	Usuario con amplias facultades de administración. Puede crear y eliminar Empresas, gestionar Roles de usuario y eliminar Usuarios. Además, puede agregar, editar y eliminar Productos desde la interfaz del sistema y posee acceso a todas las secciones del menú.
Empleado	Usuario autorizado para crear, editar y eliminar Productos, Proveedores, Personal, Marcas, Líneas y Clientes. Puede consultar la auditoría de las entidades, pero no puede gestionar Empresas ni Roles

Vendedor	Usuario autorizado para crear y consultar Productos. No puede editar ni eliminar Productos y no posee acceso a las funcionalidades de Organización ni Configuración.
Repositor	Puede crear Productos y buscarlos. No puede editar ni eliminar. Ve en el menú: Inicio y Productos. No tiene acceso a Organización ni Configuración.
Repartidor	Usuario autorizado para crear y consultar Productos. Puede editar Clientes, pero no puede registrarlos ni eliminarlos.
Cobrador	Usuario autorizado únicamente para consultar y editar Clientes. No posee acceso a la gestión de Productos, Proveedores, Personal ni Configuración .

2. Incorporación de Proveedor y conceptos relacionados al lenguaje ubicuo

Proveedor

Empresa o persona que abastece productos al negocio. Constituye una entidad con identidad y ciclo de vida propio dentro del contexto de Organización. Un Producto puede tener asociado un Proveedor principal, pero el Proveedor no pertenece al agregado Producto, ya que se gestiona independientemente y posee información propia, como CUIT, domicilio, condición frente al IVA e historial de operaciones.

Código Interno del producto

Código mediante el cual el Proveedor identifica al Producto dentro de su propio catálogo. Este código debe ser único en el sistema, por lo que no pueden existir dos productos activos que posean el mismo Código Interno.

Código de Referencia

Identificador alternativo utilizado para la búsqueda de un Producto. No posee una restricción de unicidad y permite localizar un producto mediante un código adicional al Código Interno.

3. Incorporación de las reglas de registro y habilitación de usuarios

Registro y habilitación de usuarios

El sistema permite el registro de nuevos usuarios, los cuales serán creados con el rol de mayor rango, quienes deberán contar con la información necesaria para su identificación y autenticación. Una vez registrado, cada usuario tendrá un estado que determinará si se encuentra habilitado para acceder al sistema.

Un usuario únicamente podrá autenticarse y acceder a las funcionalidades del sistema cuando se encuentre activo. Si el usuario se encuentra inactivo, su acceso deberá ser rechazado, independientemente de que las credenciales ingresadas sean correctas.

El registro podrá realizarse mediante las alternativas de autenticación admitidas por el sistema. Independientemente del mecanismo utilizado, todos los usuarios deberán cumplir las mismas condiciones de habilitación para acceder.

Una vez autenticado, el usuario podrá acceder únicamente a las funcionalidades permitidas de acuerdo con el rol que tenga asignado.